

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Felipe Esteban Luengo Bascuñán

10 de septiembre de 2026

17:14

1. Introducción

El presente informe trata el análisis experimental de algoritmos aplicados a dos problemas fundamentales: el ordenamiento de arreglos unidimensionales de números enteros y la multiplicación de matrices cuadradas. Si bien sus complejidades teóricas son conocidas, el objetivo principal de este trabajo es evaluar su comportamiento empírico en la práctica. Mediante la medición de tiempos de ejecución y consumo de memoria en distintos escenarios, se busca relacionar directamente el rendimiento práctico de las implementaciones con su complejidad teórica asintótica esperada.

2. Entorno experimental

Para la ejecución de los experimentos se utilizó una laptop ASUS Zenbook 14. Entre sus componentes de hardware cuenta con un, Intel Core Ultra 7 155H con 15 GB de memoria RAM. En cuanto a software, el sistema operativo empleado fue Fedora Linux 44 (Workstation Edition) de 64 bits. El código fuente fue compilado utilizando GCC (g++) en su versión 16.2.1, aplicando la "flag" de optimización -O3.

Respecto a los *datasets*. Para los algoritmos de ordenamiento, se crearon arreglos unidimensionales de números enteros con tamaños $N \in \{10^1, 10^3, 10^5, 10^7\}$, evaluando distribuciones ascendentes, descendentes y aleatorias, bajo los dominios D1 (rango: $[0, 9]$) y D7 (rango $[0, 10^7]$). En cuanto a la multiplicación de matrices cuadradas, se generaron matrices de dimensión $N \in \{2^4, 2^6, 2^8, 2^{10}\}$, abarcando tipos dispersos, diagonales y densos, con coeficientes en los dominios D0 (0 o 1) y D10 (rango: $[0, 9]$). Para cada combinación de parámetros se midieron tres muestras (a, b y c).

3. Resultados y Análisis

A continuación se analizan los tiempos de ejecución y el consumo de memoria de los algoritmos implementados, contrastando su rendimiento empírico con su complejidad asintótica teórica. Para establecer una base comparativa clara, el Cuadro 1 resume la complejidades esperadas para cada algoritmo.

Cuadro 1: Complejidad teórica de los algoritmos evaluados.

Algoritmo	Mejor	Promedio	Peor	Memoria	Estable	In-place
<code>std::sort</code>	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(\log N)$	No	Sí
MergeSort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	Sí	No
PatienceSort	$\mathcal{O}(N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	No	No
QuickSort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(\log N)$	No	Sí
Naive MatMul	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$	N/A	No
Strassen	$\mathcal{O}(N^{\log_2 7})$	$\mathcal{O}(N^{\log_2 7})$	$\mathcal{O}(N^{\log_2 7})$	$\mathcal{O}(N^2)$	N/A	No

3.1. Problema de Ordenamiento

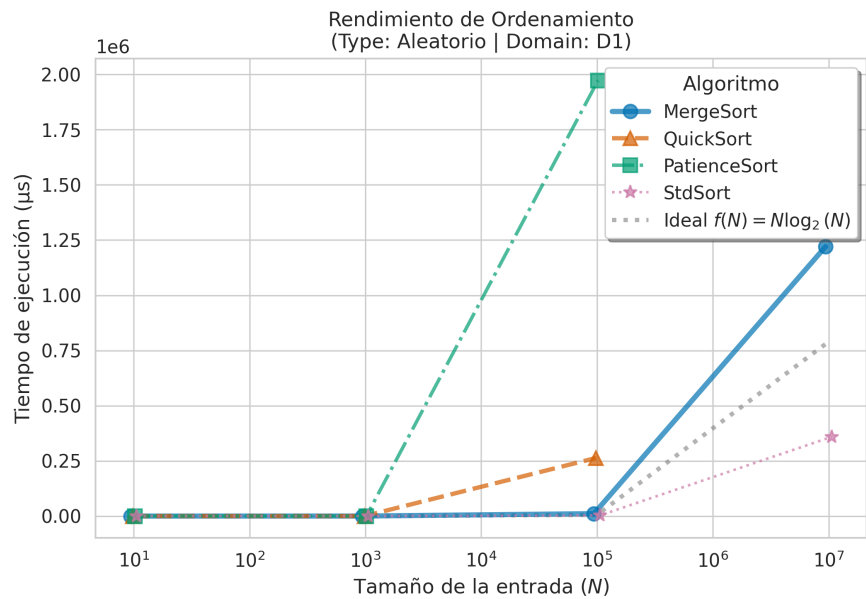


Figura 1: Tiempo de ejecución para arreglos aleatorios (Dominio D1).

Comportamiento en el caso promedio

Para arreglos aleatorios (Figura 1), los algoritmos cumplen con la teoría esperada de $\mathcal{O}(N \log N)$ (Para evidenciar se trazo una línea de tendencia ideal $N \log N$ sobre el gráfico). Sin embargo, los tiempos de ejecución difieren sustancialmente: para $N = 10^5$, MergeSort resuelve el problema en unos 11,000 μs promedio, siendo notablemente más rápido que QuickSort, el cual supera los 258,000 μs .

Esta diferencia no se explica por la complejidad teórica, sino por cómo la implementación interactúa con la memoria del computador. La implementación de QuickSort selecciona el último elemento como pivote y recorre el arreglo ejecutando incesantemente la instrucción `swap()` cada vez que encuentra un valor menor. En un arreglo desordenado, esto obliga al procesador hacer miles de sobreescrituras en memoria, frenando el rendimiento. Por otro lado, en MergeSort se gasta memoria dinámica creando subvectores temporales, su proceso de fusión avanza de manera lineal, lo que para el hardware moderno es mas llevadero.

Cuello de botella en Patience Sort

En arreglos aleatorios, Patience Sort se posiciona como el algoritmo más lento [1], tardando entre $1,900,000 \mu s$ y $2,000,000 \mu s$ para $N = 10^5$ (Dominio D1). Este bajo rendimiento, es resultado directo de 2 decisiones costosas en el código que castigan la memoria del computador. Durante la fase unir pilas (`merge_piles`) el código busca el menor número iterando por todas ellas. En un arreglo aleatorio se generan muchas pilas, lo que significa un gran gasto de tiempo comparando número por número repetitivamente. La segunda decisión o causa dentro del código es la instrucción `v.erase(v.begin() + index)`. Borrar un elemento en medio de un vector obliga a tomar todos los elementos que están a la derecha y desplazarlos hacia la izquierda, dicho reordenamiento masivo miles de veces hace que se disparen los tiempos de ejecución.

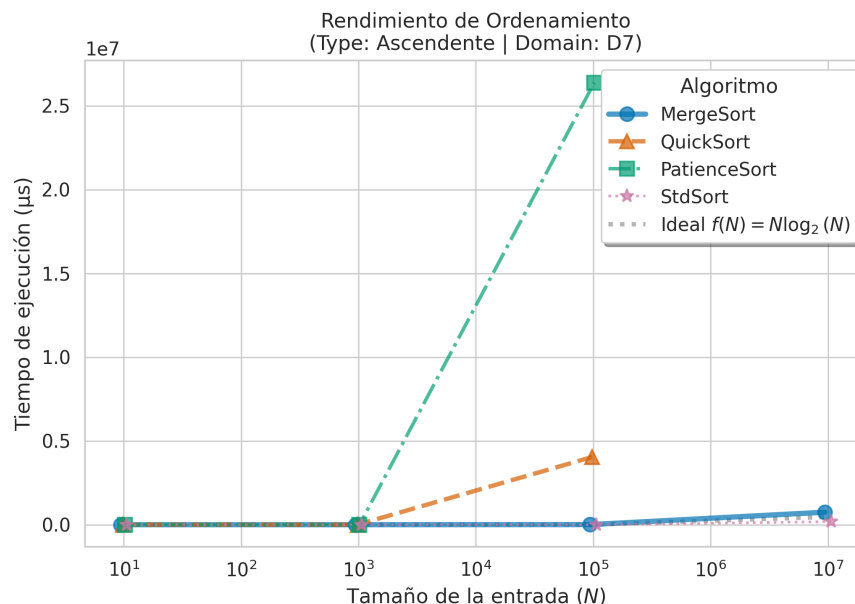


Figura 2: Tiempo de ejecución para arreglos ascendentes (Dominio D7).

Sobre QuickSort en arreglos ordenados

El mayor contraste ocurre en los arreglos ordenados (dominio ascendente, Figura 2). Aquí, QuickSort empeora drásticamente para $N = 10^5$ alcanzando $4,000,000 \mu s$ (4 segundos). Esta ineficiencia empeoró para N de tamaños mayores, por ello se limitaron los procesos hasta dicho tamaño, para tamaños mayores el programa se cerraba (Stack Overflow). Sobre las causas de este colapso. La primera es tomar el último elemento, el algoritmo espera dividir el arreglo, idealmente, en 2 mitades iguales, pero al estar el

arreglo ordenado de menor a mayor, la última posición siempre será el número más grande. Al comparar el resto de los números, todos resultan menores que el pivote, separando 1 elemento respecto a $N - 1$, en vez de dividir por la mitad ($N/2$). Al avanzar de un número a la vez, la función (`quickSort()`) se llama a sí misma 100,000 veces consecutivas, manteniendo dicha cantidad abierta en simultáneo, saturando la pila. Se podría quizás solucionar con un pivote elegido aleatoriamente (habría que probar).

QuickSort vs PatienceSort

Aunque QuickSort colapsa en arreglos ordenados demorando 4,000,000 μs , los datos muestran que PatienceSort es mucho peor, superando los 28,000,000 μs (28 segundos) para el mismo tamaño (para $N = 10^5$). Las causas. PatienceSort al recibir números ordenados de menor a mayor, el código es incapaz de apilarlos, ya que cada nuevo número es mayor al anterior, obligándolo a crear tantas pilas como elementos tenga el arreglo, las cuales están separadas por 1 solo número. Luego, al intentar unir las pilas en `merge_piles`, el programa itera por las 100,000 pilas buscando y ejecutando `v.erase()`, obligando como hemos visto a desplazar grandes bloques de memoria cientos de miles de veces seguidas. Por otro lado, en QuickSort se repite la ineficiencia por elegir el último elemento como pivote, descrito anteriormente.

`std::sort` no falla

Todas las gráficas y plots generados evidencian que `std::sort` opera de la manera más eficiente independiente del dominio y distribución específica del arreglo, resolviendo el mismo problema $N = 10^5$ en apenas 1,000 μs promedio. La explicación es la arquitectura híbrida de *Introsort* [2].

El compilador inicia el proceso utilizando QuickSort para aprovechar su velocidad en memoria caché, pero lleva un registro interno de cuántas veces se ha llamado la función a sí misma. Si detecta que la recursión es profunda, como en un arreglo ascendente por ejemplo, cambia automáticamente a otro algoritmo (como HeapSort) que garantiza terminar el trabajo eficientemente sin consumir más memoria de la necesaria. Por eso está en la librería estándar.

3.2. Multiplicación de Matrices

Al evaluar la multiplicación de matrices (Figura 3 y Figura 4) nos encontramos con una discrepancia entre la teoría y los resultados prácticos. Teóricamente, el algoritmo de Strassen ($\mathcal{O}(N^{2.81})$) es superior al algoritmo Naive ($\mathcal{O}(N^3)$) [3]. Pero, en los tamaños evaluados, Naive es superior. Para el caso de matrices densas de $N = 1024$, Naive finalizó en 2 segundos, mientras que Strassen llegó a los 130 segundos. Esta diferencia se comprende si observamos la implementación de cada uno y cómo tratan la memoria. Naive opera mediante 3 for anidados, multiplicando y acumulando el resultado de manera directa. No requiere memoria extra durante todo el cómputo, esto se refleja en sus 0KB de consumo adicional, posibilitando aprovechar al máximo la memoria caché. Por otro lado, Strassen para lograr una multiplicación menos, debe realizar múltiples sumas y restas dividiendo el problema. Ello implica instanciar arreglos bidimensionales temporales en cada uno de los niveles recursivos. En el tamaño $N = 1024$, esta creación masiva de submatrices satura el sistema operativo, provocando un consumo de memoria de 93,272KB.

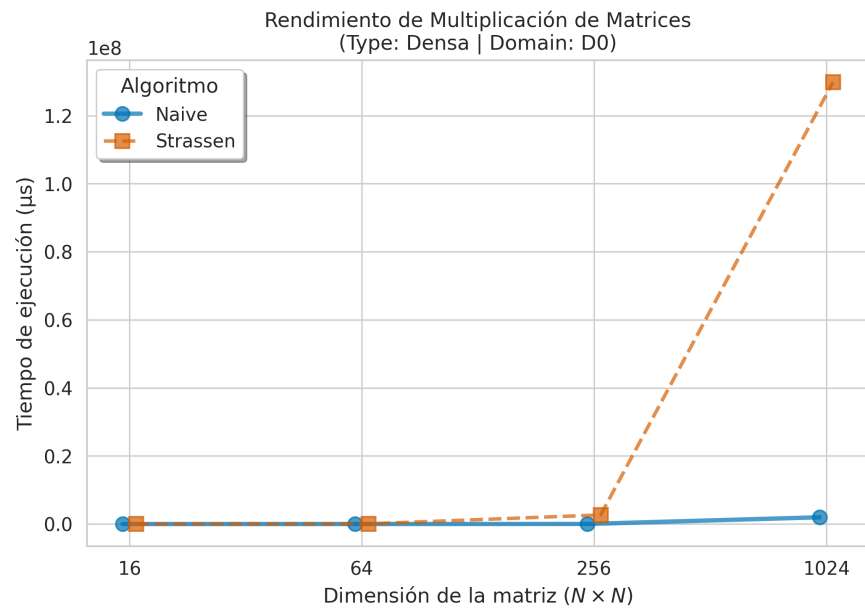


Figura 3: Tiempo de ejecución para matrices densas (Dominio D0).

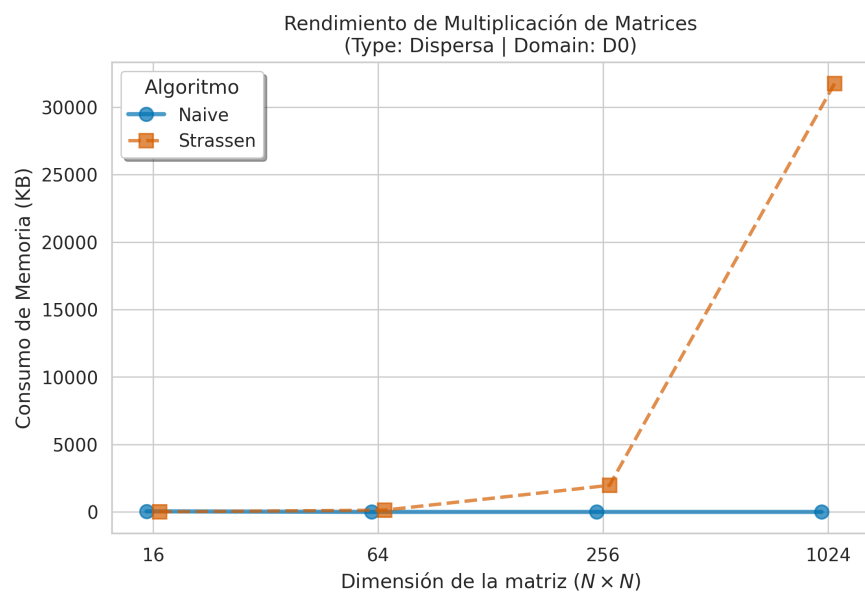


Figura 4: Consumo de memoria para matrices dispersas.

4. Conclusiones

El presente análisis experimental demuestra empíricamente que la notación asintótica no es el único factor determinante en el rendimiento real de un algoritmo. Si bien las cotas teóricas orientan el comportamiento esperado, son las decisiones de implementación y su interacción a bajo nivel con la memoria del computador las que dictan el verdadero costo computacional.

En definitiva, la elección de un algoritmo en ingeniería de software debe ir más allá de su comportamiento asintótico e incluir obligatoriamente el tamaño real de la entrada, la distribución de los datos y, sobre todo, el costo físico que las estructuras temporales y los patrones de acceso generan sobre la memoria del computador.

Referencias

- [1] David Aldous y Persi Diaconis. «Longest increasing subsequences: from patience sorting to the Baik-Deift-Johansson theorem». En: *Bulletin of the American Mathematical Society* 36.4 (1999), págs. 413-432.
- [2] C++ community. *std::sort* - *cppreference.com*. <https://en.cppreference.com/w/cpp/algorithm/sort>. 2026.
- [3] Thomas H Cormen et al. *Introduction to Algorithms*. 4th. MIT press, 2022.